Jarosław Kuchta

Jakość Oprogramowania

Jakość oprogramowania w metodykach zwinnych (Agile)

Wady klasycznych metod zapewnienia jakości

- Duży narzut na dokumentowanie
- Późne uzyskiwanie konkretnych rezultatów
- Trudność w odpowiednio wczesnym definiowaniu wymagań
- Utrata łączności z klientem (użytkownikiem) w fazie implementacji
- Brak pewności co do uzyskanej jakości

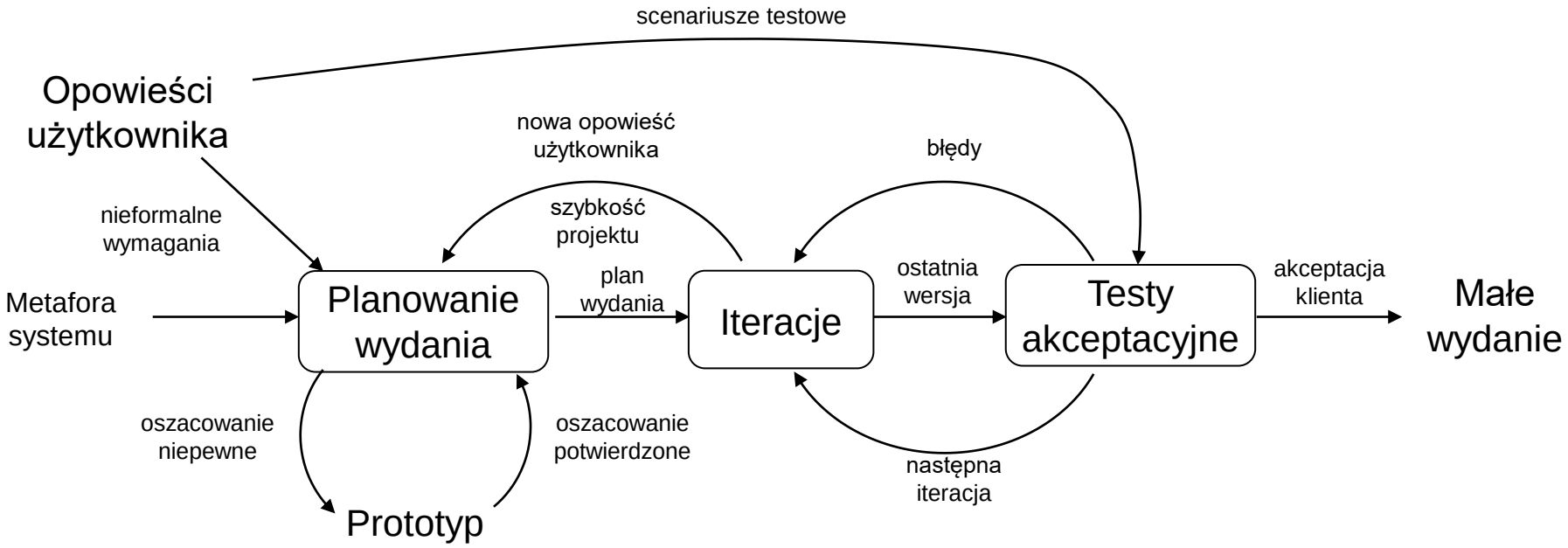
Założenia
Agile Programming (1/2)

1. Najwyższym priorytetem jest zadowolenie klienta, które można zapewnić przez szybkie i ciągłe dostarczanie działającego oprogramowania.
2. Dopuszczalne są zmiany wymagań nawet w późnym stadium projektu. Projekt musi być dopasowywany do zmieniających się wymagań i warunków na korzyść klienta.
3. Działające oprogramowanie jest dostarczane często, co kilka tygodni lub co kilka miesięcy. Im częściej tym lepiej.
4. Potrzebna jest bliska, codzienna współpraca między przedstawicielami klienta i zespołem projektowym.
5. Projekty są oparte o odpowiednio zmotywowanych deweloperów, którym trzeba zapewnić środowisko pracy i zaufać, że wykonają swoją pracę.
6. Najbardziej odpowiednią i efektywną metodą zbierania informacji przez zespół projektowy jest bezpośrednia rozmowa.

Założenia
Agile Programming (2/2)

7. Działające oprogramowanie jest podstawową miarą postępu prac.
8. Proces opracowywania oprogramowania powinien być stale podtrzymywany przez sponsorów, deweloperów i użytkowników.
9. Potrzebne jest stałe zwracanie uwagi na techniczną doskonałość i dobre projektowanie
10. Upraszczanie ma znaczenie zasadnicze
11. Najlepsze wymagania, projekty i struktury pochodzą od samoorganizujących się zespołów.
12. W regularnych odstępach zespół projektowy zastanawia się nad tym, jak zwiększyć swoją efektywność, następnie odpowiednio zmienia i dopasowuje swoje sposoby postępowania.

Model cyklu pracy (Extreme Programming)



Co to jest jakość w Agile?

Najwyższym priorytetem jest zadowolenie klienta



dobry produkt = zadowolenie klienta ?

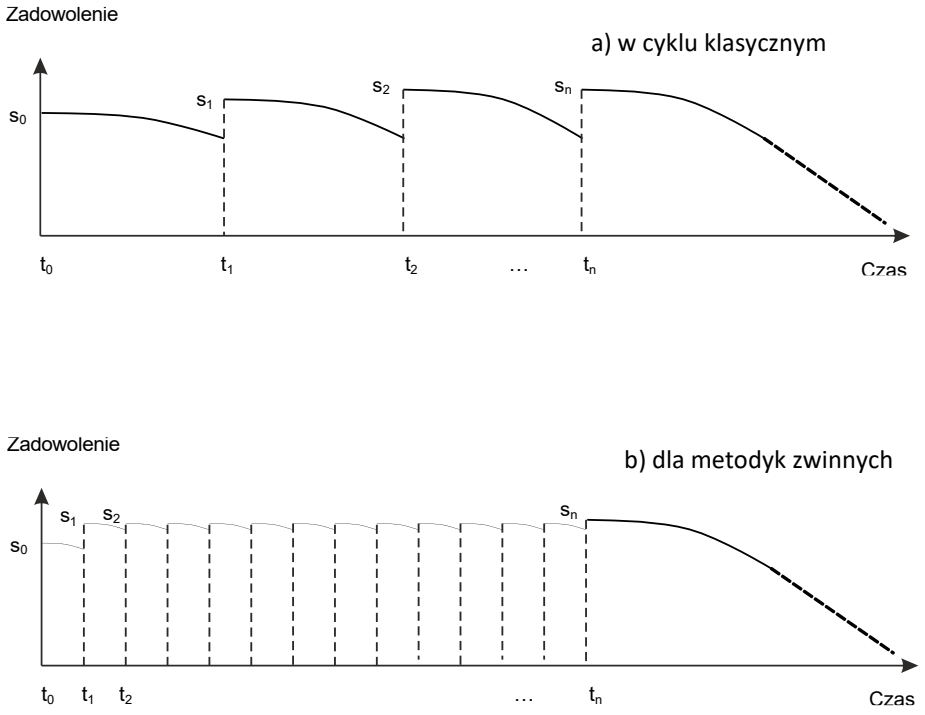


Tak chciał klient, więc tak zrobiliśmy.
Nie rozumiemy, dlaczego klient ma do nas pretensje.

Model Kano wymagań klienta

- Klient ma zawsze trzy rodzaje wymagań:
  - Wymagania, które zna i o których mówi wykonawcy
    - Spełnienie tych wymagań jest konieczne, aby klient był zadowolony (ale może nie być wystarczające).
  - Wymagania, które zna, lecz o których nie mówi wykonawcy
    - Spełnienie tych wymagań też jest konieczne, aby klient był zadowolony, bo jak nie będą spełnione, to będzie niezadowolony.
  - Wymagania, których nie zna
    - Spełnienie tych wymagań powoduje, że klient jest mile zaskoczony.
    - Jeśli te wymagania są konieczne dla zapewnienia jakości produktu wymaganej przez rynek lub przepisy, to ujawnienie braku ich spełnienia spowoduje, że klient będzie niezadowolony.

Model zmiany satysfakcji klienta w czasie



Praktyki zwinne (XP)

- Praktyki planowania i pozyskiwania wymagań
  - Klient na miejscu (w zespole)
  - Opowieści użytkownika
  - Małe, częste wydania, krótkie iteracje
  - Planowanie wydania, planowanie iteracji
- Praktyki zarządzania
  - Otwarta przestrzeń pracy, spotkania na stojąco
  - Stały rytm pracy, unikanie pracy w nadgodzinach
  - Pomiar szybkości projektu przez opowieści
  - Rotacja personelu
  - Naprawa procesu (elastyczność metody)
- Praktyki związane z projektowaniem
  - Metafora systemu
  - Prostota projektu
  - Unikanie dodawania funkcjonalności na przyszłość
  - Rozwiązania próbne
  - Sesje projektowania
  - Refaktoryzacja
- Praktyki związane z kodowaniem i integracją
  - Standardy kodowania
  - Programowanie w parach
  - Kolektywna własność kodu
  - Sekwencyjna integracja
  - Częsta (nieustanna) integracja
  - Wydzielone środowisko wdrożenia
- Praktyki związane z testowaniem
  - Kodowanie testów przez implementacją (TDD)
  - Pokrycie kodu testami jednostkowymi
  - Obowiązkowe zaliczanie testów jednostkowych przez wydaniem kodu
  - Tworzenie nowych testów dla zabezpieczenia przed nawrotem błędu
  - Testy akceptacyjne na podstawie opowieści użytkownika

Problemy z klientem (1)

- Założenia:
  - Klient (jego przedstawiciel) jest cały czas dostępny, pracuje w bliskim kontakcie z zespołem, najlepiej aby stanowił część zespołu.
  - Klient podaje tylko ogólne wymagania w opowieściach użytkownika, a uszczegóławia je przez codzienny kontakt z zespołem.
- Realia:
  - sceptycyzm i niezrozumienie roli klienta w projektach zwinnych
  - duża odległość od klienta do zespołu utrudniająca komunikację (np. klient i zespół działają w różnych strefach czasowych)
  - brak zaangażowania klienta spowodowany rozmaitymi czynnikami (np. niskim priorytetem danego projektu u klienta)
  - duża organizacja kliencka wykazująca przywiązanie do tradycyjnych metod
  - gdy wynagrodzenie jest z góry ustalone (tak jak w tradycyjnych kontraktach), to klient nie ma motywacji finansowej do angażowania się w pracę zespołu
  - niekompetencja przedstawiciela klienta – brak wystarczającej wiedzy lub możliwości decyzyjnych u osoby wyznaczonej przez klienta na swojego przedstawiciela

Problemy z klientem (2)

- Konsekwencje
  - presja na ustalenie warunków (zakresu terminu, wynagrodzenia) ze strony klienta
  - problemy ze zbieraniem wymagań od klienta i uzyskaniem wyjaśnień co do szczegółów wymagań
  - problemy z ustaleniem priorytetów wymagań przez klienta
  - problemy z uzyskaniem informacji zwrotnej od klienta po dostarczeniu wydania
  - opóźnienia i spadek wydajności pracy zespołu
  - straty finansowe, a w ostateczności fiasko projektu.

Strategie przeciwdziałania

- **Dążenie do *zmiany postawy klienta*** (podkreślanie zalet Agile na tle wad metod tradycyjnych)
- **Zapewnienie różnych *opcji kontraktowych*** (np. zakup kilku kolejnych wydań zamiast finansowania całego projektu z góry, wybór funkcjonalności, klauzula o wypowiedzeniu umowy)
- **Dodawanie *marginesu czasowego*** do planowanego harmonogramu (aby uwzględnić zmienność czasu implementacji)
- **Obniżanie *priorytetów opowieści*** użytkowników, które czekają na wyjaśnienia klienta,
- **Stosowanie pojęcia *„gotowości”* opowieści *do implementacji*** (opowieść „gotowa” ma określone cele biznesowe, wynik i szczegóły implementacji niezbędne do oszacowania pracochłonności)
- **Wstępna *ocena ryzyka*** związanego z brakiem zaangażowania klienta
- **przydzielanie *właścicieli do opowieści***, a nie tylko do całego produktu
- **Zastępczy *przedstawiciel klienta*** (*proxy*) działający jako pośrednik między klientem a zespołem (w tej roli analityk biznesowy)
- **Silny nacisk na *regularne pokazy*** (demonstracje) działającego oprogramowania (zbieranie informacji zwrotnych)
- **Ukrywanie metod zwinnych** stosowanych przez zespół przed klientem

W metodyce Scrum rolę klienta pełni właściciel produktu!

Rola właściciela produktu prowadzi do wypalenia zawodowego

Opowieści użytkownika a specyfikacja wymagań

Klasyczne wymagania	Opowieści użytkownika
Powinny być kompletne	Z założenia nie są kompletne
Powinny być stabilne	Z założenia mogą się zmieniać
Powinny być precyzyjne	Są ogólnikowe, a precyzowane w czasie implementacji
Powinny być wystarczające do implementacji	Powinny być wystarczające do oszacowania pracochłonności
Są funkcjonalne i niefunkcjonalne	Dotyczą zasadniczo tylko funkcjonalności
Powinny być spójne i niesprzeczne	Mogą zmieniać poprzednie wymagania

Wymagania przedstawiciela klienta mogą się różnić od wymagań użytkowników

Brak analizy sytuacji wyjątkowych może się ujawnić po dłuższym czasie

Problem stałego rytmu pracy

- Kolejne iteracje planuje się na podstawie szacowania opowieści użytkownika
- Założenie nienaruszalne – stała długość iteracji
- Naruszenie harmonogramu – przeniesienie części działań do następnej iteracji
- Brak czasu na zajmowanie się jakością
- Mało czasu – kodowanie z długiem technicznym
- Potrzebne dodatkowe iteracje na realizację zaległości

Często dług techniczny w ogóle nie jest odrabiany!

Otwarta przestrzeń robocza

- Założenia:
  - Praca w otwartej przestrzeni ułatwia komunikację w zespole, stymuluje wzajemną inspirację, ułatwia rozwiązywanie problemów.
  - W trakcie codziennych spotkań na stojąco członkowie zespołu wybierają i uzgadniają realizację zadań.
- Realia
  - Brak możliwości spotkań osobistych przy pracy zdalnej.
  - Utrudniona komunikacja na odległość przy różnicach w strefach czasowych.
  - Problemy w zrozumieniu (różne języki, idiomy, gwary).
  - Problemy kulturowe (np. brak akceptacji równości płci).

Rotacja personelu (brak dokumentacji)

- Założenia:
  - Wiedza o projekcie znajduje się w głowach ludzi, a nie w dokumentacji.
  - Aby nie dopuścić do nadmiernej koncentracji wiedzy u pojedynczych osób trzeba zmieniać przydziały pracowników do różnych zespołów.
  - Praca w parach daje możliwość wprowadzenia w szczegóły młodszego pracownika przez starszego.
- Realia:
  - Praca w parach jest nieefektywna finansowo.
  - Część wiedzy jest tracona wskutek naturalnej, dużej rotacji personelu.
  - Odejście kluczowego programisty może pogrążyć projekt.
  - Konieczność ciągłego podtrzymywania procesu (dla utrzymania wiedzy).
  - Brak możliwości podjęcia projektu ponownie po niedługiej przerwie.

Prostota projektu

- Założenia:
  - Projekty proste są łatwiejsze w realizacji i utrzymaniu niż złożone.
  - Prostota oznacza TUBE:
    - Testability – testowalność – moduły powinny być możliwe do testowania jednostkowego, a cały system – do testowania akceptacyjnego,
    - Understandability – zrozumiałość – zespół musi być w stanie ogarnąć zrozumieniem cały projekt,
    - Browsability – wyszukiwalność – programista musi być w stanie łatwo znaleźć w projekcie to, czego potrzebuje (np. przez wyszukiwanie nazw),
    - Explainability – objaśnialność – zespół musi być w stanie wytłumaczyć nowo przyjętej osobie, o co chodzi w projekcie.
- Realia:
  - Nie każdy projekt da się zrealizować jako prosty.
  - Nawet proste projekty z czasem stają się złożone.
  - W projektach złożonych następuje odejście od metodyk zwinnych na rzecz hybrydowych.

Unikanie dodawania funkcjonalności na przyszłość

- Założenia:
  - 80% użytkowników korzysta z 20% funkcji (reguła Pareto).
  - Nie ma sensu dodawanie funkcjonalności uzupełniających, które nie będą wykorzystywane.
- Realia:
  - To narusza teorię wymagań Kano!
  - To się nie sprawdza przy frameworkach.
  - Większość frameworków działa dobrze tylko przy bardzo rygorystycznym przestrzeganiu wskazówek do stosowania („how to”). Zawodzą przy najmniejszej próbie innego zastosowania.
  - Ryzyko niedopasowania do oprogramowania użytkownika jest duże – zagrożenie dla projektu!

Stosowanie standardów kodowania

- Założenia:
  - Najlepsi, dobrze opłacani programiści znają dobrze standardy kodowania, dzięki którym minimalizują popełnianie błędów.
- Realia:
  - Liczba najlepszych programistów na rynku jest mocno ograniczona. Nawet najlepsza płaca nie zagwarantuje, że tacy trafią do zespołu.
  - Firmy muszą się pogodzić z tym, że pracują u nich programiści słabo wykształceni, z niewielkim doświadczeniem.
  - Powinno się zapewnić szkolenie dla takich programistów.
  - Programiści po szkoleniu i nabraniu doświadczenia zgłaszają żądania wyższej płacy i odchodzą do konkurencji.

Programowanie w parach

- Założenia:
  - Programowanie w parach daje szansę nadzorowania pracy młodszego programisty przez starszego, a przez to minimalizuje okazje do błędów.
- Realia:
  - Programowanie w parach jest nieefektywne finansowo.
  - W praktyce nie jest stosowane.

Kodowanie testów przed implementacją (Test Driven Development)

- Założenia:
  - Pisząc testy przed implementacją funkcji umożliwia napisanie bezbłędnego kodu – od razu musi spełniać warunki testu.
- Realia
  - To podejście może powodować, że programy są pisane „pod testy”, a nie „pod użytkownika”.

Testy jednostkowe

- Założenia:
  - Dobrze napisane testy muszą pokrywać wszystkie ścieżki wykonania.
  - Napisany kod przed przekazaniem do wykorzystania musi przejść wszystkie testy.
- Realia:
  - Pełne pokrycie kodu testami jest w praktyce niemożliwe.
  - Testy muszą być wykonywane nie przez autorów, lecz przez innych. Zaniedbanie tego wymagania prowadzi do braku wiarygodności testów.

Testy akceptacyjne

- Założenia:
  - Testy akceptacyjne są tworzone na podstawie *opowieści użytkownika*.
  - Implementacja opowieści nie jest zakończona dopóki nie przejdzie wszystkich testów akceptacyjnych.
  - To klient powinien być odpowiedzialny za warunki testów akceptacyjnych i ocenę ich wyników.
- Realia:
  - Nie ma pewności, czy opowieści przekazywane przez klienta wyrażają rzeczywiste sposoby użycia.
  - Testy akceptacyjne są z reguły pisane, wykonywane i oceniane przez testerów (u wykonawcy).
  - Użytkownicy końcowi nie przeprowadzają regularnych testów. Jedynie sprawdzają wybrane działania.
  - Ujawnienie się błędów u użytkowników końcowych powoduje zawyżone koszty.
  - Użytkownicy końcowi nie przeprowadzają testów regresyjnych (zakładają, że to co działało, działa w dalszym ciągu).

Warunki zapewnienia jakości w metodykach Agile

1. Dobra, codzienna współpraca (komunikacja) z klientami (użytkownikami końcowymi) w całym procesie wytwarzania.
2. Formułowanie celów wydań na podstawie rzeczywistych wymagań klientów, a nie tylko ich opowieści.
3. Zapewnienie dobrego przepływu informacji w zespole i między wszystkimi zainteresowanymi stronami – najlepiej praca w bezpośrednim kontakcie, a jeśli tego nie można zapewnić, to utrzymywanie częstych kontaktów,
4. Znajomość i stosowanie odpowiednich standardów pracy inżynierskiej.
5. Rygorystyczne przeprowadzanie testów jednostkowych przed opublikowaniem każdego wydania.
6. Poddawanie wydań przed rozpowszechnieniem testom akceptacyjnym przeprowadzanym przez wybranych użytkowników.

Wady (zagrożenia) metodyk zwinnych

- Uzależnienie procesu od ludzi – zmienność składu zespołu – zagrożenie stabilności projektu
- Brak dokumentacji – brak możliwości powrotu do projektu po dłuższej przerwie – zagrożenie dla rozwoju
- Brak analizy sytuacji wyjątkowych – możliwość przeoczenia problemów i popełnienia poważnych błędów projektowych – zagrożenie awaryjności w czasie eksploatacji
- Zespół cały czas wprowadza zmiany – pracuje w trybie odpowiadającym fazie pielęgnacji metod klasycznych – to generuje wysokie koszty – zagrożenie przekroczenia budżetu i braku opłacalności

Zastosowanie

- Przy prostych projektach
- W małych, skoncentrowanych organizacjach
- W projektach o niskim poziomie ryzyka
- W projektach zapewniających możliwość nieustannej pielęgnacji produktu

Źródła

- Kent Beck: *Extreme Programming Explained: Embrace Change*, Addison-Wesley, 1999
- http://www.xprogramming.com
- http://www.extremeprogramming.org
- http://www.agile.com
- http://www.pairprogramming.com
- http://www.martinfowler.com
- http://www.xp123.com
- http://www.thoughtworks.com
- http://www.rolemodelsoftware.com
- http://www.industriallogic.com/xp
- http://www.agilealliance.org